ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



HỆ ĐIỀU HÀNH

Project 1

FILE & PROCESS MANAGEMENT TRÊN XV6

Nhóm: 6
Sinh viên thực hiện: Nguyễn Quốc Đạt – 24120282
Nguyễn Thành Dự – 24120288
Nguyễn Đức Duy – 24120294

Thành phố Hồ Chí Minh, 11/2026

Mục lục

1 Tổng quan **1**

1.1 Mức độ hoàn thành . . . 1

1.2 Phân công công việc . . . 1

1.3 Môi trường thực hiện . . . 1

2 Tìm hiểu về File **2**

2.1 Field `ofile[]` của `struct proc` và `file[]` của `struct ftable` . . . 2

2.2 Trace system call `open` . . . 3

2.3 So sánh cách dùng file descriptor trong `open()`, `dup()`, `fork()` . . . 4

3 Tìm hiểu về Process **6**

3.1 Khái niệm process trong xv6 . . . 6

3.2 Các field `pid`, `state`, `context` của `struct proc` . . . 6

3.2.1 `pid` – định danh process . . . 7

3.2.2 `state` – trạng thái process . . . 7

3.2.3 `context` – ngữ cảnh kernel thread . . . 8

3.3 Trace system call `fork` . . . 9

3.3.1 Từ user vào kernel . . . 9

3.3.2 Hàm `fork()` trong kernel . . . 9

3.3.3 Lần chạy đầu tiên của process con . . . 10

3.4 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()` . . . 10

3.4.1 Khi nào CPU chuyển sang process khác . . . 10

3.4.2 `scheduler()` . . . 11

3.4.3 `sched()` . . . 11

3.4.4 `yield()` . . . 12

3.4.5 Toàn bộ quá trình chuyển process . . . 12

3.4.6 Khóa `p->lock` được giữ qua `swtch` . . . 12

3.4.7 Liên hệ với phần cài đặt `sysinfo` . . . 13

4 Cài đặt system call `sysinfo` **14**

4.1 Yêu cầu và ý tưởng . . . 14

4.2 Cấu trúc dữ liệu (`kernel/sysinfo.h`) . . . 14

4.3 Các thay đổi trong mã nguồn . . . 15

4.4 Luồng xử lý trong kernel . . . 15

4.5 Các vấn đề kỹ thuật và cách giải quyết . . . 15

4.6 Kiểm thử 16
4.7 Chạy với chương trình thật 18
4.8 Khó khăn và hạn chế 20

Tài liệu tham khảo **21**

1 Tổng quan

1.1 Mức độ hoàn thành

STT	Yêu cầu	Mức độ	Mục
4.1 – Tìm hiểu và báo cáo			
1	Field `ofile[]` của `struct proc`, `file[]` của `struct ftable`	100%	2.1
2	Trace system call `open`	100%	2.2
3	So sánh cách dùng file descriptor trong `open()`, `dup()`, `fork()`	100%	2.3
4	Field `pid`, `state`, `context` của `struct proc`	100%	3.2
5	Trace system call `fork`	100%	3.3
6	Cơ chế lập lịch `scheduler()`, `sched()`, `yield()`	100%	3.4
4.2 – Cài đặt			
7	System call `sysinfo`: danh sách file đang mở	100%	4
8	System call `sysinfo`: danh sách process khác `UNUSED`	100%	4
9	Chương trình test `user/sysinfotest.c` (10 nhóm test)	100%	4.6

Bảng 1: Mức độ hoàn thành các yêu cầu

Ngoài yêu cầu của đề, nhóm viết thêm lệnh `sysinfo` (`user/sysinfo.c`) để quan sát trạng thái hệ thống khi chạy các chương trình thật trong shell (mục 4.7).

1.2 Phân công công việc

MSSV	Họ tên	Công việc	Hoàn thành
24120282	Nguyễn Quốc Đạt	*(cần điền)*	100%
24120288	Nguyễn Thành Dự	*(cần điền)*	100%
24120294	Nguyễn Đức Duy	*(cần điền)*	100%

Bảng 2: Phân công công việc

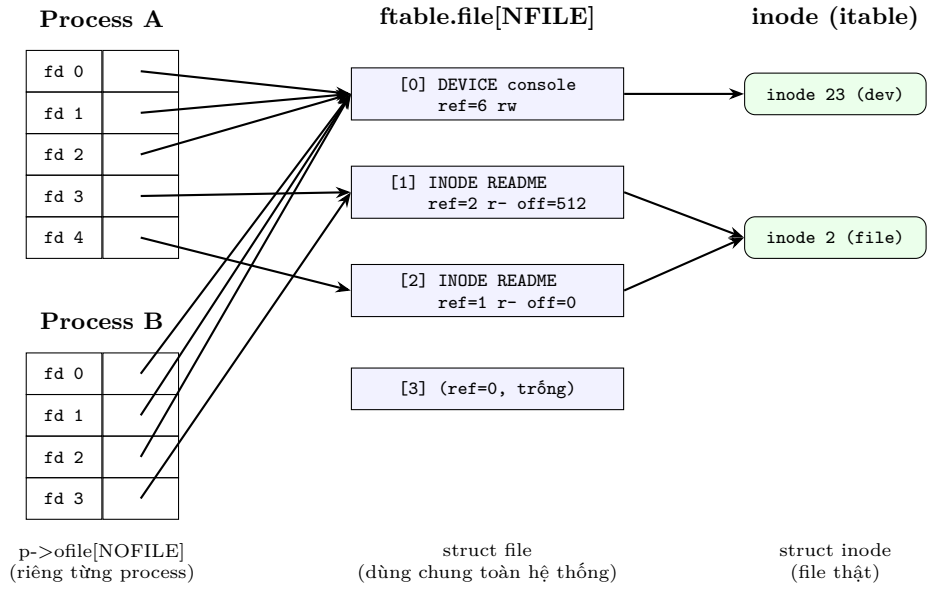
1.3 Môi trường thực hiện

- Mã nguồn: xv6-labs-2024 (MIT 6.1810), nhánh `util`.
- Hệ điều hành: Ubuntu Linux; trình biên dịch `riscv64-linux-gnu-gcc`; giả lập `qemu-system-riscv64` với 3 CPU (`-smp 3`).
- Biên dịch và chạy: `make qemu`; thoát QEMU bằng `Ctrl-a x`.

2 Tìm hiểu về File

2.1 Field `ofile[]` của `struct proc` và `file[]` của `struct ftable`

File descriptor (fd) là một số nguyên nhỏ mà process dùng để chỉ tới một đối tượng có thể đọc/ghi: file thường, thư mục, thiết bị (console) hoặc pipe. Chương trình user chỉ thấy con số này; bên trong kernel, mỗi fd được giải mã qua ba tầng (Hình 1).



Hình 1: Ba tầng giải mã file descriptor. B là con của A, được tạo bằng `fork` sau khi A mở README lần thứ nhất (fd 3) nên dùng chung ô [1]; sau đó A mở README lần nữa (fd 4) và nhận ô [2] riêng.

`p->ofile[NOFILE].` Mảng 16 con trỏ `struct file*` trong `struct proc`; **fd chính là chỉ số** trong mảng này. Ô bằng 0 nghĩa là fd đó chưa dùng. Mảng là riêng của từng process, nên fd 3 của process A và fd 3 của process B có thể trỏ tới hai file khác nhau. Theo quy ước, fd 0, 1, 2 là stdin, stdout, stderr. Field này thuộc nhóm "riêng tư" của `struct proc`, chỉ chính process đó sửa (trong `open`, `dup`, `close`, `fork`, `exit`) nên không cần `p->lock`.

`ftable.file[NFILE].` Bảng file đang mở của toàn hệ thống (`kernel/file.c`), gồm 100 phần tử `struct file` và một spinlock `ftable.lock`. Mỗi phần tử ứng với *một lần mở* (một lần gọi `open` hoặc `pipe`):

Field	Ý nghĩa
`type`	`FD_NONE` (ô trống), `FD_PIPE`, `FD_INODE` (file, thư mục), `FD_DEVICE` (thiết bị như console).
`ref`	Số con trỏ đang trỏ vào ô này (từ `ofile[]` của mọi process). `ref == 0` nghĩa là ô trống. Được bảo vệ bởi `ftable.lock`.
`readable, writable`	Mode mở: được đọc / được ghi hay không.
`pipe`	Con trỏ tới `struct pipe` nếu là pipe.
`ip`	Con trỏ tới inode nếu là file, thư mục hoặc thiết bị.
`off`	Vị trí đọc/ghi hiện tại (chỉ với `FD_INODE`).
`major`	Số hiệu thiết bị (chỉ với `FD_DEVICE`), dùng tra bảng `devsw[]`.

Bảng 3: Các field của `struct file`

Các hàm quản lý bảng:

- `filealloc()`: dưới `ftable.lock`, tìm ô có `ref == 0`, đặt `ref = 1` và trả về ô đó.
- `filedup(f)`: tăng `f->ref`.
- `fileclose(f)`: giảm `f->ref`; khi về 0 thì đặt `type = FD_NONE` và giải phóng tài nguyên bên dưới (`pipeclose` hoặc `iput`).

Vì sao cần tầng `ftable` ở giữa. Nếu `ofile[]` trỏ thẳng vào inode thì không có chỗ lưu offset riêng cho từng lần mở. Tầng `struct file` cho phép offset vừa được *chia sẻ* (giữa các fd tạo bởi `dup` hoặc `fork`) vừa được *tách biệt* (giữa hai lần `open` khác nhau trên cùng một file).

2.2 Trace system call `open`

1. **User space:** `open(path, omode)` nạp `SYS_open` vào `a7`, gọi `ecall`. Kernel đi qua `uservec` → `usertrap()` → `syscall()` → `sys_open()` (`kernel/sysfile.c`), giống như `fork` ở mục 3.3.
2. **Lấy tham số:** `argint(1, &omode)` lấy mode; `argstr(0, path, MAXPATH)` chép chuỗi đường dẫn từ bộ nhớ user vào kernel (qua `copyinstr`, dùng page table của process).
3. **Bắt đầu transaction:** `begin_op()`, vì `open` có thể ghi lên đĩa (tạo file mới).
4. **Tìm hoặc tạo inode:**
   - Có `O_CREATE`: `create(path, T_FILE, 0, 0)` tìm thư mục cha, nếu file chưa có thì `ialloc` một inode mới và `dirlink` thêm entry vào thư mục. Trả về inode đã khóa.

- Không có `O_CREATE`: `namei(path)` duyệt đường dẫn từng thành phần (`dirlookup`) để lấy inode, rồi `ilock(ip)` (đọc inode từ đĩa nếu chưa có trong bộ nhớ). Nếu là thư mục mà mở để ghi thì trả về lỗi.

5. **Kiểm tra thiết bị:** nếu `T_DEVICE` thì `major` phải hợp lệ ($0 \leq \text{major} < \text{NDEV}$).
6. **Cấp `struct file`:** `filealloc()` lấy một ô trống trong `ftable` (`ref = 1`). Nếu bảng đầy (100 ô), trả về -1.
7. **Cấp fd:** `fdalloc(f)` duyệt `p->ofile[]` từ 0, gán `f` vào **ô trống có chỉ số nhỏ nhất** và trả về chỉ số đó. Nếu process đã dùng đủ 16 fd, `fileclose(f)` trả lại ô `ftable` và `open` trả về -1.
8. **Điền `struct file`:** `type` là `FD_DEVICE` (kèm `major`) hoặc `FD_INODE` (kèm `off = 0`); `ip = ip`; `readable = !(omode & O_WRONLY)`; `writable = (omode & O_WRONLY) || (omode & O_RDWR)`.
9. **`O_TRUNC`:** nếu có và là file thường thì `itrunc(ip)` xóa nội dung.
10. **Kết thúc:** `iunlock(ip)` (vẫn giữ tham chiếu inode qua `f->ip`), `end_op()`, trả về `fd`.

Bước 7 (chọn fd nhỏ nhất) là cơ sở của chuyển hướng I/O trong shell: với lệnh `sysinfo -f > out`, shell `close(1)` rồi `open("out", ...)`, nên file `out` nhận đúng fd 1 (kết quả ở Hình 7).

2.3 So sánh cách dùng file descriptor trong `open()`, `dup()`, `fork()`

	`open(path, mode)`	`dup(fd)`	`fork()`
fd mới	1 fd, nhỏ nhất còn trống	1 fd, nhỏ nhất còn trống	Process con có bản sao toàn bộ `ofile[]`, cùng số fd
`struct file`	**Cấp ô mới** (`filealloc`)	Dùng lại ô cũ	Dùng lại các ô của cha
`ref`	Ô mới có `ref = 1`	`ref + 1`	Mỗi ô đang mở: `ref + 1` (`filedup`)
Offset	Riêng, bắt đầu từ 0	Dùng chung với fd gốc	Dùng chung giữa cha và con
Mode	Theo tham số `mode`	Giống fd gốc	Giống của cha
Phạm vi	Trong process gọi	Trong process gọi	Giữa hai process
Ứng dụng	Mở file	Chuyển hướng I/O, ví dụ `init` tạo fd 1, 2 từ fd 0	Shell chia sẻ console và pipe cho lệnh con

Bảng 4: So sánh file descriptor trong `open`, `dup`, `fork`

Tóm lại, chỉ `open` (và `pipe`) tạo ra một `struct file` mới. `dup` và `fork` chỉ tạo thêm con trỏ tới `struct file` đã có, nên các fd đó *dùng chung offset*: ví dụ cha và con cùng ghi vào một file sau `fork` thì dữ liệu được nối tiếp chứ không ghi đè lên nhau. Ngược lại, hai lần `open` cùng một file cho hai `struct file` độc lập, chỉ dùng chung inode.

Các tính chất này được kiểm chứng bằng `sysinfo` trong test `test_openclose`, `test_dup`, `test_fork` (mục 4.6): hai lần `open` cho hai `idx` khác nhau cùng `inum`; `dup` cho cùng `idx` với `ref = 3` và offset chung; sau `fork` các fd của con có cùng `idx` với cha và `ref` tăng 1.

3 Tìm hiểu về Process

3.1 Khái niệm process trong xv6

Process là *đơn vị cách ly* của xv6: một process không thể đọc hay phá bộ nhớ, CPU, file descriptor của process khác, cũng không thể phá kernel. Để làm được điều này, kernel tạo cho mỗi chương trình hai ảo giác:

- **Bộ nhớ riêng (address space):** mỗi process có một page table riêng (`p->pagetable`) ánh xạ địa chỉ ảo bắt đầu từ 0 sang bộ nhớ vật lý. Process này không nhìn thấy bộ nhớ của process khác.
- **CPU riêng (thread):** kernel luân phiên CPU giữa các process. Khi một process không chạy, các thanh ghi của nó được lưu lại và khôi phục khi nó được chạy tiếp.

Trong xv6 mỗi process có đúng một address space và một thread. Mỗi process có **hai stack**: user stack dùng khi chạy code user và kernel stack (`p->kstack`, 1 trang) dùng khi vào kernel qua system call hoặc interrupt. Kernel stack được tách riêng và được bảo vệ để kernel vẫn chạy đúng kể cả khi chương trình user làm hỏng user stack của nó.

Kernel quản lý process bằng một mảng cố định `struct proc proc[NPROC]` (`NPROC = 64`). Không có cấp phát động: tạo process nghĩa là tìm một ô có `state == UNUSED` và điền thông tin vào (`allocproc()`).

3.2 Các field `pid`, `state`, `context` của `struct proc`

`struct proc` (`kernel/proc.h`) chứa toàn bộ trạng thái kernel của một process. Các field được chia nhóm theo khóa bảo vệ chúng (Bảng 5).

Khóa bảo vệ	Field	Lý do
`p->lock`	`state`, `pid`, `chan`, `killed`, `xstate`	Được process khác và scheduler trên CPU khác đọc/ghi.
`wait_lock` (toàn cục)	`parent`	Process chưa biết cha mình là ai cho tới khi lấy được khóa, nên không thể dùng khóa riêng của cha.
Không cần khóa	`kstack`, `sz`, `pagetable`, `trapframe`, `context`, `ofile`, `cwd`, `name`	Chỉ chính process đó sử dụng.

Bảng 5: Các nhóm field của `struct proc`

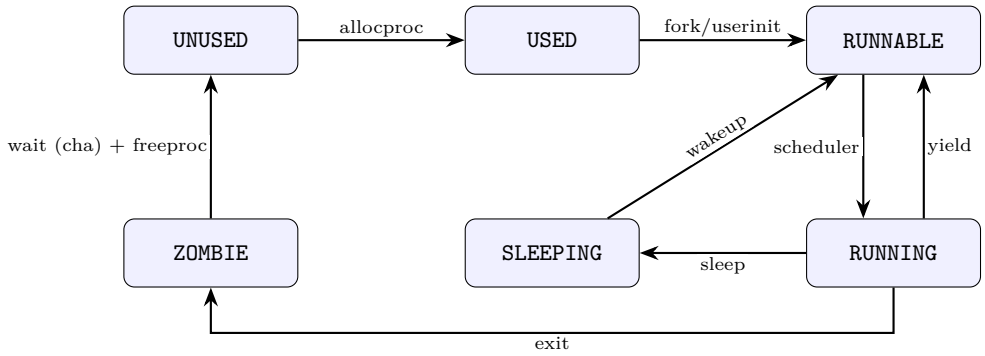
3.2.1 `pid` – định danh process

`pid` là số nguyên dương duy nhất, cấp bởi `allocpid()` khi `allocproc()` tìm được ô trống. Hàm này tăng biến toàn cục `nextpid` dưới khóa `pid_lock`, nên hai CPU tạo process cùng lúc không bao giờ nhận trùng pid. Process đầu tiên (`init`) có pid 1. pid không được tái sử dụng dù ô `proc[]` có được dùng lại.

pid được dùng làm giá trị trả về của `fork()` cho process cha, giá trị trả về của `wait()`, và tham số của `kill(pid)`. Vì `kill` và `wait` đọc pid của process khác nên field này phải được bảo vệ bởi `p->lock`: `kill` cần chắc chắn process nạn nhân không thoát và bị cấp phát lại giữa lúc kiểm tra pid và lúc đặt cờ `killed`.

3.2.2 `state` – trạng thái process

`state` có kiểu `enum procstate {UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE}`, cho biết ô trong bảng đang trống, đang được dựng, đang chờ sự kiện, sẵn sàng chạy, đang chạy trên một CPU, hay đã kết thúc nhưng chưa được cha thu dọn. Hình 2 và Bảng 6 mô tả các chuyển trạng thái.



Hình 2: Sơ đồ chuyển trạng thái của process trong xv6

Chuyển	Nguyên nhân
`UNUSED` → `USED`	`allocproc()` tìm thấy ô trống: gán pid, cấp trang trapframe, tạo page table, đặt `context.ra = forkret`.
`USED` → `RUNNABLE`	Cuối `fork()` (hoặc `userinit()`), sau khi đã dựng xong toàn bộ process con.
`RUNNABLE` → `RUNNING`	`scheduler()` chọn process để chạy.
`RUNNING` → `RUNNABLE`	`yield()`: hết lượt do timer interrupt.
`RUNNING` → `SLEEPING`	`sleep(chan)`: chờ sự kiện (đọc pipe, `wait`, đọc console, …); `p->chan` ghi lại kênh đang chờ.
`SLEEPING` → `RUNNABLE`	`wakeup(chan)` từ process khác hoặc interrupt.
`RUNNING` → `ZOMBIE`	`exit()`: đóng file, giao con cho `init`, đánh thức cha, lưu `xstate`.
`ZOMBIE` → `UNUSED`	`wait()` của cha lấy mã thoát rồi gọi `freeproc()`.

Bảng 6: Các chuyển trạng thái và hàm gây ra chúng

Trạng thái `ZOMBIE` cần thiết vì hai lý do: process cha có thể chưa gọi `wait` khi con thoát nên kernel phải giữ lại mã thoát; và một process không thể tự giải phóng kernel stack mà nó đang chạy trên đó. Nếu cha thoát trước con, `exit()` gọi `reparent()` để giao các con cho `init`, process luôn gọi `wait` trong vòng lặp, nên mọi process con đều có người thu dọn.

Trạng thái `USED` là trạng thái trung gian khi process đang được dựng. Scheduler chỉ chạy process `RUNNABLE`, nên một process đang dựng dở bị "che" khỏi scheduler cho tới khi `fork()` đặt nó thành `RUNNABLE` ở bước cuối cùng.

3.2.3 `context` – ngữ cảnh kernel thread

`struct context` lưu 14 thanh ghi của *kernel thread* của process: `ra`, `sp` và các thanh ghi callee-saved `s0`–`s11`. Nó được dùng bởi hàm assembly `swtch(old, new)` (`kernel/swtch.S`): lưu các thanh ghi hiện tại vào `old`, nạp các thanh ghi từ `new`, rồi `ret`.

- `swtch` chỉ lưu các thanh ghi callee-saved, vì trình biên dịch C đã lưu các thanh ghi caller-saved lên stack trước khi gọi hàm.
- `swtch` không lưu `pc`, mà lưu `ra`. Khi `ret`, CPU nhảy tới `ra` vừa nạp, tức là ngay sau lệnh gọi `swtch` trước đây của thread kia.
- Vì `sp` cũng được nạp lại, CPU chuyển sang chạy trên stack của thread kia.

Mỗi CPU cũng có một `context` trong `struct cpu`, là ngữ cảnh của *scheduler thread* của CPU đó. Process chuyển CPU bằng `swtch(&p->context, &mycpu()->context)` và scheduler chuyển CPU cho process bằng `swtch(&c->context, &p->context)`.

Cần phân biệt `context` với `trapframe` (Bảng 7):

	`p->trapframe`	`p->context`
Lưu gì	Toàn bộ thanh ghi *user* (`epc`, `sp`, `a0`–`a7`, ...)	`ra`, `sp`, `s0`–`s11` của *kernel thread*
Khi nào	User → kernel (system call, interrupt)	Kernel thread → scheduler (`swtch`)
Ai lưu	`uservec` trong `trampoline.S`	`swtch.S`
Mục đích	Quay về đúng lệnh trong chương trình user	Quay về đúng chỗ trong code kernel

Bảng 7: So sánh `trapframe` và `context`

Khi `allocproc()` tạo process mới, nó đặt `context.ra = forkret` và `context.sp = p->kstack + PGSIZE`. Do đó lần đầu scheduler `swtch` vào process mới, CPU "quay về" đầu hàm `forkret()` trên kernel stack rỗng của process đó.

3.3 Trace system call `fork`

3.3.1 Từ user vào kernel

1. Chương trình user gọi `fork()`. Hàm này được sinh bởi `user/usys.pl`: nạp số hiệu `SYS_fork` vào thanh ghi `a7` rồi thực thi lệnh `ecall`.

2. `ecall` chuyển CPU sang supervisor mode và nhảy tới `uservec` (`kernel/trampoline.S`). `uservec` lưu toàn bộ thanh ghi user vào `p->trapframe`, chuyển sang page table kernel và kernel stack của process, rồi gọi `usertrap()`.

3. `usertrap()` (`kernel/trap.c`) thấy `scause == 8` (system call), tăng `trapframe->epc` thêm 4 để khi trở về sẽ chạy lệnh sau `ecall`, bật interrupt và gọi `syscall()`.

4. `syscall()` đọc `a7`, tra bảng `syscalls[]` và gọi `sys_fork()`, hàm này chỉ gọi `fork()` trong `kernel/proc.c`. Giá trị trả về được ghi vào `trapframe->a0`.

3.3.2 Hàm `fork()` trong kernel

1. **Cấp phát process:** `allocproc()` duyệt `proc[]`, tìm ô `UNUSED`, gán pid mới, chuyển sang `USED`, cấp trang trapframe, tạo page table rỗng (chỉ có trampoline và trapframe) và khởi tạo `context`. Hàm trả về với `np->lock` đang được giữ. Nếu hết ô hoặc hết bộ nhớ, `fork` trả về -1.

2. **Sao chép bộ nhớ:** `uvmcopy()` cấp trang mới và chép toàn bộ bộ nhớ user (code, data, stack, heap) của cha sang con, rồi đặt `np->sz = p->sz`. Hai process có nội dung bộ nhớ giống nhau nhưng nằm ở các trang vật lý khác nhau.

3. **Sao chép thanh ghi user:** `*(np->trapframe) = *(p->trapframe)`. Do đó khi con trở về user space, nó tiếp tục chạy ngay sau lệnh `ecall` của `fork`, giống hệt cha.

4. **Giá trị trả về của con:** `np->trapframe->a0 = 0`. Đây là điểm khác duy nhất giữa thanh ghi của cha và con, giải thích vì sao `fork` "trả về hai lần": 0 ở con, pid của con ở cha.

5. **Chia sẻ file đang mở:** với mỗi `p->ofile[i]` khác 0, gán `np->ofile[i]` bằng `filedup(p->ofile[i])`. `filedup` chỉ tăng `ref` của `struct file` chứ không mở file mới, nên cha và con dùng chung offset. Thư mục hiện tại được chia sẻ bằng `idup(p->cwd)`.

6. **Tên:** chép `p->name` sang `np->name`.

7. **Ghi nhận cha:** nhả `np->lock`, lấy `wait_lock`, đặt `np->parent = p`.

8. **Cho phép chạy:** lấy lại `np->lock`, đặt `np->state = RUNNABLE`. Chỉ từ lúc này scheduler mới có thể chọn process con.

9. Trả về pid của con. Giá trị này đi qua `trapframe->a0` của cha.

3.3.3 Lần chạy đầu tiên của process con

Khi một CPU chọn process con, `scheduler()` gọi `swtch(&c->context, &np->context)`. Vì `context.ra = forkret`, CPU nhảy vào `forkret()`. Hàm này nhả `p->lock` (khóa được scheduler lấy trước khi `swtch`, xem mục 3.4.6), lần đầu tiên thì khởi tạo file system, rồi gọi `usertrapret()`. `usertrapret()` và `userret` nạp lại thanh ghi user từ trapframe đã sao chép ở bước 3 và thực thi `sret`. Process con trở về user space ngay sau lệnh `ecall`, với `a0 = 0`.

Process cha trở về theo đường bình thường: `syscall()` ghi pid vào `a0`, `usertrap()` gọi `usertrapret()` và cha tiếp tục chạy. Thứ tự cha hay con chạy trước phụ thuộc vào scheduler.

3.4 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()`

3.4.1 Khi nào CPU chuyển sang process khác

xv6 chuyển process trong hai trường hợp:

- **Tự nguyện:** process gọi một system call phải chờ (`read`, `wait`, `sleep`, …) và đi vào `sleep()`.
- **Bắt buộc:** timer interrupt định kỳ. Khi `devintr()` trả về 2 (timer), `usertrap()` hoặc `kerneltrap()` gọi `yield()`, ép process đang tính toán nhường CPU.

3.4.2 `scheduler()`

Mỗi CPU, sau khi khởi động xong, gọi `scheduler()` và không bao giờ quay lại. Hàm này là một vòng lặp vô hạn chạy trên stack riêng của CPU (scheduler thread):

1. Bật interrupt (tránh deadlock khi mọi process đều đang chờ I/O).
2. Duyệt lần lượt `proc[0..NPROC-1]`. Với mỗi `p`: lấy `p->lock`; nếu `p->state == RUNNABLE` thì đặt `p->state = RUNNING`, `c->proc = p`, rồi `swtch(&c->context, &p->context)` để chạy `p`.
3. Khi `p` nhường CPU, `swtch` "trả về" trong scheduler. Đặt `c->proc = 0`, nhả `p->lock` và xét process tiếp theo.
4. Nếu duyệt hết bảng mà không có process nào `RUNNABLE`, CPU thực thi `wfi` để nghỉ tới interrupt tiếp theo.

Chính sách này là **round robin**: các process được chạy lần lượt, không có độ ưu tiên. Scheduler chạy trên stack riêng thay vì stack của một process, vì nếu dùng stack của process thì process đó có thể được đánh thức và chạy trên CPU khác, dẫn tới hai CPU dùng chung một stack.

3.4.3 `sched()`

`sched()` là *điểm duy nhất* một kernel thread nhường CPU cho scheduler. Trước khi gọi `swtch(&p->context, &mycpu()->context)`, nó kiểm tra bốn điều kiện, vi phạm bất kỳ điều kiện nào sẽ gây `panic`:

- đang giữ `p->lock`;
- chỉ giữ đúng một khóa (`mycpu()->noff == 1`), vì giữ thêm khóa khác qua `swtch` có thể gây deadlock;
- `p->state != RUNNING`, tức là người gọi đã đổi trạng thái;
- interrupt đang tắt.

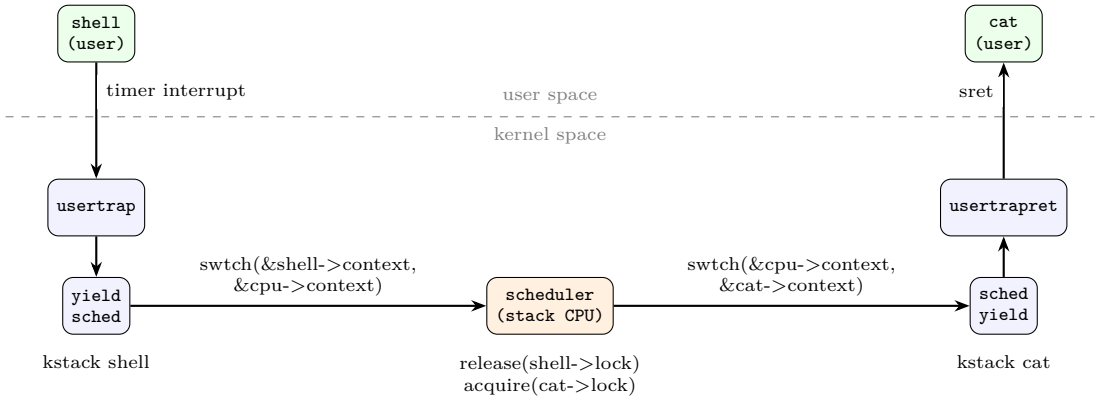
`sched()` cũng lưu và khôi phục `intena`, vì đây là thuộc tính của kernel thread chứ không phải của CPU. Mọi nơi muốn nhường CPU (`yield`, `sleep`, `exit`) đều làm theo cùng một mẫu: lấy `p->lock`, đổi `p->state`, gọi `sched()`.

3.4.4 `yield()`

`yield()` là cách nhường CPU khi hết lượt: lấy `p->lock`, đặt `p->state = RUNNABLE`, gọi `sched()`. Khi process được chọn chạy lại, `sched()` trả về và `yield()` nhả `p->lock`. Process ở trạng thái `RUNNABLE` nên sẽ được scheduler chọn lại ở một vòng duyệt sau.

3.4.5 Toàn bộ quá trình chuyển process

Hình 3 mô tả việc chuyển từ process `shell` sang process `cat` trên một CPU. Mỗi lần chuyển gồm **hai** lần `swtch`: process → scheduler và scheduler → process mới. `sched` và `scheduler` luân phiên chuyển quyền điều khiển cho nhau, nên chúng được gọi là các *coroutine*.



Hình 3: Chuyển CPU từ `shell` sang `cat`

3.4.6 Khóa `p->lock` được giữ qua `swtch`

Thông thường thread nào lấy khóa thì chính thread đó nhả khóa. Trong xv6, `p->lock` được giữ xuyên qua `swtch`:

- `yield()` của `shell` lấy `shell->lock`, nhưng `scheduler()` mới là nơi nhả nó.
- `scheduler()` lấy `cat->lock`, nhưng `yield()` của `cat` (hoặc `forkret()` nếu `cat` vừa được tạo) mới nhả nó.

Lý do: nếu `shell->lock` được nhả ngay sau khi đặt `state = RUNNABLE` nhưng trước khi `swtch` lưu xong ngữ cảnh, scheduler của một CPU khác có thể chọn `shell` và chạy nó trên chính kernel stack mà CPU hiện tại vẫn đang dùng. Giữ khóa tới khi `swtch` hoàn tất đảm bảo mỗi process chỉ chạy trên đúng một CPU tại một thời điểm.

3.4.7 Liên hệ với phần cài đặt `sysinfo`

Để trả về "number of scheduling events" theo yêu cầu 4.2, nhóm thêm field `nsched` vào `struct proc`:

- Khởi tạo bằng 0 trong `allocproc()`, vì ô `proc[]` được dùng lại cho process mới.
- Tăng 1 trong `scheduler()` ngay sau `p->state = RUNNING` và trước `swtch`. Đây là nơi duy nhất một process được cấp CPU, nên `nsched` đếm chính xác số lần process được lập lịch. Tại điểm này scheduler đang giữ `p->lock`, nên hai CPU không thể cùng tăng biến.

Khi đọc thông tin process, `sysinfo` lấy `wait_lock` trước rồi mới lấy `p->lock`, cùng thứ tự với `wait()` và `exit()`, để đọc `parent` đúng mà không gây deadlock.

4 Cài đặt system call `sysinfo`

4.1 Yêu cầu và ý tưởng

`sysinfo(struct sysinfo *info)` thu thập thông tin hệ thống đang chạy:

- Các file đang mở trong toàn hệ thống, tức các ô `ftable.file[]` có `ref > 0`: số lượng và với mỗi file là type, mode, reference count, danh sách process tham chiếu.
- Các process có `state != UNUSED` trong `proc[]`: số lượng và với mỗi process là pid, state, pid của cha, số lần được lập lịch, danh sách file đang mở.

Danh sách "process tham chiếu tới file" được tính ngược: process nào có một ô `ofile[]` trỏ tới `struct file` đó. Để người dùng đối chiếu được hai danh sách, mỗi file mang chỉ số `idx` trong `ftable`, và mỗi fd của process mang chỉ số `fileidx` của ô nó trỏ tới.

4.2 Cấu trúc dữ liệu (`kernel/sysinfo.h`)

`struct fileinfo` – một file đang mở	
`idx`	Vị trí trong `ftable.file[]`
`type, readable, writable, ref`	Lấy trực tiếp từ `struct file`
`inum, itype, major, off`	Số inode, loại inode (file/dir/device), thiết bị, offset; -1 nếu không áp dụng (ví dụ pipe)
`nfd`	Tổng số fd (của mọi process) trỏ vào file này
`npid, pids[NPROC]`	Danh sách pid các process tham chiếu (mỗi pid một lần)
`struct procinfo` – một process khác `UNUSED`	
`pid, state, ppid, name`	Định danh, trạng thái, pid của cha (0 nếu không có), tên
`nsched`	Số lần được scheduler chọn chạy (field mới trong `struct proc`)
`nofile, fds[], fileidx[]`	Các fd đang mở và chỉ số ô `ftable` tương ứng
`struct sysinfo`	
`nfile, files[NFILE]`	Số file đang mở và mảng `fileinfo`
`nproc, procs[NPROC]`	Số process và mảng `procinfo`

Bảng 8: Các cấu trúc trả về cho user

4.3 Các thay đổi trong mã nguồn

File	Thay đổi
`kernel/sysinfo.h`	(mới) Định nghĩa `fileinfo`, `procinfo`, `sysinfo`.
`kernel/proc.h`	Thêm field `nsched` vào `struct proc`.
`kernel/proc.c`	`allocproc`: `nsched = 0`. `scheduler`: `nsched++` trước `swtch`. Thêm `procfilerefs()` và `procsysinfo()`.
`kernel/file.c`	Thêm `fileindex()` và `filesysinfo()`.
`kernel/sysproc.c`	Thêm `sys_sysinfo()`.
`kernel/syscall.h, syscall.c`	`SYS_sysinfo = 23`; thêm vào bảng `syscalls[]` và `syscallnames[]` (để lệnh `trace` của Lab0 in được tên).
`kernel/defs.h`	Prototype của 4 hàm mới.
`user/user.h, user/usys.pl`	Khai báo `struct sysinfo;`, `int sysinfo(struct sysinfo *);` và stub `entry("sysinfo")`.
`user/sysinfotest.c`	(mới) Chương trình test.
`user/sysinfo.c`	(mới) Lệnh in trạng thái hệ thống.
`Makefile`	Thêm `_sysinfotest`, `_sysinfo` vào `UPROGS`.

Bảng 9: Danh sách file thay đổi

4.4 Luồng xử lý trong kernel

1. `sys_sysinfo()` lấy địa chỉ user `addr` bằng `argaddr(0)`.
2. Gọi `filesysinfo(addr + offsetof(files))`. Với mỗi ô của `ftable.file[]`: lấy `ftable.lock`; bỏ qua nếu `ref == 0`; chép type, mode, ref, thông tin inode vào biến cục bộ `fileinfo`; gọi `procfilerefs(f)` để duyệt `proc[]` và ghi lại pid các process có `ofile[]` trỏ tới `f`; nhả khóa; `copyout` phần tử đó vào `files[n]` của user.
3. Gọi `procsysinfo(addr + offsetof(procs))`. Với mỗi `p` trong `proc[]`: lấy `wait_lock` rồi `p->lock`; bỏ qua nếu `UNUSED`; chép pid, state, `parent->pid`, nsched, name và danh sách fd (kèm `fileindex()` của từng file) vào biến cục bộ `procinfo`; nhả khóa; `copyout` vào `procs[n]`.
4. `copyout` hai giá trị `nfile`, `nproc` vào đầu struct và trả về 0. Nếu bất kỳ `copyout` nào thất bại (địa chỉ không hợp lệ, không ghi được hoặc vượt `MAXVA`) thì trả về -1.

4.5 Các vấn đề kỹ thuật và cách giải quyết

Struct quá lớn so với stack. `struct sysinfo` có kích thước khoảng 40 KB (100×300 B $+64 \times 164$ B), trong khi kernel stack chỉ có 4 KB. Vì vậy kernel không dựng toàn bộ struct

rồi `copyout` một lần, mà chỉ giữ *một* `fileinfo` hoặc `procinfo` trên stack và `copyout` thẳng vào đúng vị trí trong struct của user (`addr + offsetof(files) + n * sizeof(fileinfo)`). Ở phía user, user stack chỉ có 8 KB nên chương trình test khai báo struct là biến `static` toàn cục.

Đồng bộ trên nhiều CPU. xv6 chạy trên 3 CPU, nên trong lúc `sysinfo` đọc, process khác có thể đang `open`, `close`, `fork` hay `exit`. Nhóm dùng các khóa sẵn có theo đúng vai trò của chúng:

- `ftable.lock` được giữ trong suốt lúc đọc một ô *và* dò `ofile[]`, nên `ref` và danh sách pid luôn nhất quán (`nfd` không bao giờ vượt `ref`).
- `p->lock` bảo vệ `state`, `pid`; `wait_lock` bảo vệ `parent`.
- Thứ tự khóa: phần file là `ftable.lock` → `p->lock`; trong xv6 không có chỗ nào giữ `p->lock` rồi mới lấy `ftable.lock` (`exit` đóng file trước khi lấy `p->lock`), nên không gây deadlock. Phần process là `wait_lock` → `p->lock`, cùng thứ tự với `wait()` và `exit()`.
- Mọi khóa được nhả trước khi `copyout`.

Đếm số lần lập lịch. `nsched` được tăng trong `scheduler()` ngay trước `swtch` vào process, là nơi duy nhất một process được cấp CPU, và lúc đó đang giữ `p->lock` (chi tiết ở mục 3.4).

4.6 Kiểm thử

`user/sysinfotest.c` gọi `sysinfo()` trước và sau mỗi thao tác rồi so sánh hai kết quả. Mỗi lần gọi còn được kiểm tra các bất biến chung: `ref > 0`, `nfd` $\leq$ `ref`, process `ZOMBIE` không giữ file, mọi `fileidx` của process đều có trong danh sách file và file đó liệt kê ngược lại pid của process.

Test	Kịch bản và kết quả mong đợi
`basic`	Thấy `init` (pid 1, ppid 0, SLEEPING), `sh` là cha của test, test ở trạng thái RUNNING; fd 0/1/2 cùng trỏ một file console (`FD_DEVICE`, major 1, mode `rw`).
`bad addresses`	Địa chỉ 0 (trang text chỉ đọc), địa chỉ vượt `MAXVA`, struct tràn qua cuối heap đều trả về -1; gọi đúng sau đó vẫn thành công.
`open / close`	`nfile` tăng/giảm 1; mode đúng với `O_RDONLY`, `O_WRONLY`, `O_RDWR`; mở thư mục có `itype` dir; offset là 10 sau khi ghi 10 byte; hai lần `open` cùng file cho hai ô khác nhau cùng inode.
`dup`	Không tạo ô mới; 3 fd cùng `idx`, `ref = nfd = 3`; offset dùng chung; `close` giảm `ref`.
`pipe`	`nfile` tăng 2; một đầu chỉ đọc, một đầu chỉ ghi; không có inode.
`fork / exit / wait`	Con có `ppid` đúng, SLEEPING khi chờ pipe, cùng `idx` với cha, `ref` tăng (console tăng 3); con exit nhưng chưa wait thì là ZOMBIE, không còn file, `ref` giảm; sau `wait` con biến mất.
`many processes`	10 process con: `nproc` tăng 10; đầu đọc pipe có `ref = npid = 11`; đầu ghi chỉ cha giữ.
`scheduling counter`	`nsched` tăng ít nhất 5 sau 5 lần `sleep(1)`; process con mới có `nsched` nhỏ.
`fd limit`	Mở tới khi lỗi: `nofile` đúng bằng `NOFILE` $= 16$.
`ftable full`	12 process con mở file tới khi lỗi: `nfile` đúng bằng `NFILE` $= 100$; sau khi con thoát, các ô được giải phóng.

Bảng 10: Các nhóm test trong `sysinfotest`

Kết quả: tất cả test đều đạt, chạy hai lần liên tiếp trong cùng một lần khởi động vẫn đạt (không rò rỉ file hay process). `usertests -q` của xv6 cũng đạt toàn bộ, cho thấy các thay đổi không ảnh hưởng tới phần còn lại của kernel.

```
$ sysinfotest
test basic:
  open files: 1
    idx type   mode ref inum itype off  pids
    0   device rw   9   23   dev   0    1 2 3
  processes: 3
    pid ppid state  nsched name        fd->ftable
    1   0    sleep  13     init        0->0 1->0 2->0
    2   1    sleep  12     sh          0->0 1->0 2->0
    3   2    run    23     sysinfotest 0->0 1->0 2->0
  OK
test bad addresses:
  OK
test open / close / modes:
  OK
test dup:
  OK
test pipe:
  OK
test fork / exit / wait:
  ...
  OK
test many processes:
  OK
test scheduling counter:
  nsched 68 -> 73
  OK
test per-process fd limit (NOFILE):
  OK
test system-wide file table full (NFILE):
  nfile=100 (NFILE=100)
  OK
sysinfotest: ALL TESTS PASSED
```

Hình 4: Kết quả chạy `sysinfotest` trên QEMU

4.7 Chạy với chương trình thật

Lệnh `sysinfo [-f | -p]` (`user/sysinfo.c`) gọi system call và in bảng file (`-f`), bảng process (`-p`) hoặc cả hai. Kết hợp với shell, lệnh này cho thấy trực quan các khái niệm ở phần tìm hiểu.

Pipeline. Với `cat README | grep the | sysinfo` (Hình 5), shell tạo hai pipe (ô 1–2 và 3–4). stdout của `cat` (fd 1) trỏ vào ô 2 (đầu ghi pipe thứ nhất), stdin của `grep` trỏ vào ô 1 (đầu đọc), stdout của `grep` trỏ vào ô 4, stdin của `sysinfo` trỏ vào ô 3. `cat` mở README ở fd 3 (ô 5, đã đọc tới offset 2048). Console (ô 0) có `ref = 16` vì được mọi process kế thừa qua `fork`. Các process `sh` (pid 28, 30) là các bước fork trung gian của shell.

```
$ cat README | grep the | sysinfo
OPEN FILES: 6
IDX  TYPE    MODE  REF  INUM  ITYPE  OFF   PIDS
0    device  rw    16   23    dev    0     1 2 28 29 30 31 32
1    pipe    r-    2    -     -      -     30 31
2    pipe    -w    1    -     -      -     29
3    pipe    r-    1    -     -      -     32
4    pipe    -w    1    -     -      -     31
5    inode   r-    1    2     file   2048  29

PROCESSES: 7
PID  PPID  STATE  NSCHED  NAME     FD->FILE
1    0     sleep  13      init     0->0 1->0 2->0
2    1     sleep  15      sh       0->0 1->0 2->0
28   2     sleep  1       sh       0->0 1->0 2->0
29   28    sleep  8       cat      0->0 1->2 2->0 3->5
30   28    sleep  1       sh       0->1 1->0 2->0
31   30    run    9       grep     0->1 1->4 2->0
32   30    run    9       sysinfo  0->3 1->0 2->0
```

Hình 5: `sysinfo` ở cuối một pipeline

Zombie và reparent. Chương trình `zombie` của xv6 fork một con rồi `sleep`; con thoát ngay. Chạy `zombie &; sysinfo -p` (Hình 6): pid 36 ở trạng thái ZOMBIE và không còn fd nào, chờ cha (pid 35) gọi `wait`. Pid 35 có `ppid = 1` vì process shell chạy nền đã tạo nó đã thoát, nên nó được `reparent` cho `init`.

```
$ zombie &; sysinfo -p
PROCESSES: 5
PID  PPID  STATE   NSCHED  NAME     FD->FILE
1    0     sleep   14      init     0->0 1->0 2->0
2    1     sleep   17      sh       0->0 1->0 2->0
33   2     run     2       sysinfo  0->0 1->0 2->0
36   35    zombie  1       zombie
35   1     sleep   6       zombie   0->0 1->0 2->0
```

Hình 6: Process ZOMBIE và process được giao cho `init`

Chuyển hướng output. `sysinfo -f > out` rồi `cat out` (Hình 7): file `out` (inode 24) chiếm ô 1 với mode chỉ ghi, và chính là fd 1 của `sysinfo`, đúng như phân tích ở bước 7 của mục 2.2. Lệnh `trace` của Lab0 với mask 2^{23} cũng in được tên syscall mới.

```
$ sysinfo -f > out
$ cat out
OPEN FILES: 2
IDX  TYPE    MODE  REF  INUM  ITYPE  OFF  PIDS
0    device  rw    8    23    dev    0    1 2 37
1    inode   -w    1    24    file   0    37
$ trace 8388608 sysinfo -p
syscall sysinfo -> 0
...
```

Hình 7: Chuyển hướng output vào file và `trace` syscall `sysinfo`

4.8 Khó khăn và hạn chế

- **Kích thước struct:** ban đầu dự định dựng `struct sysinfo` trong kernel rồi `copyout` một lần, nhưng 40 KB vượt quá kernel stack 4 KB. Giải pháp là `copyout` từng phần tử.
- **`printf` của xv6 không hỗ trợ `%c`:** bản đầu của bảng in ra ký tự rác ở cột mode, phải đổi sang in chuỗi.
- **Test bảng file đầy:** bản đầu dùng 8 process con nhưng chỉ đạt 93 file, vì mỗi con đã có sẵn 5 fd nên chỉ mở thêm được 11 file. Đã tăng lên 12 process con.
- **Đọc `ip->type` không khóa:** trường này được bảo vệ bởi sleep-lock của inode, nhưng không thể ngủ khi đang giữ spinlock `ftable.lock`. Vì file đang mở (`ref > 0`) nên inode luôn hợp lệ và loại của nó không đổi, nên việc đọc là an toàn.
- **Ảnh chụp tại một thời điểm:** kết quả chỉ đúng tại lúc gọi; các process khác có thể thay đổi trạng thái ngay sau đó. Mỗi phần tử là nhất quán, nhưng giữa các phần tử với nhau thì không được bảo đảm (không khóa toàn bộ hệ thống trong suốt lời gọi).
- Các field `pids[]` và `fds[]` có kích thước cố định theo `NPROC` và `NOFILE`, nên struct khá lớn dù đa số phần tử để trống.

Tài liệu tham khảo

[1] R. Cox, F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, revision 4, 2024 (`docs/book-xv6-riscv-rev4.pdf`).

[2] Mã nguồn xv6-riscv, MIT 6.1810 Fall 2024: `git://g.csail.mit.edu/xv6-labs-2024`.